

AFLLY TOOL EXPERIMENT

Name:

- Disha Lokesh (PES1UG24AM093)
- Dhriti Kamani (PES1UG24AM085)

Class: 3B (AIML)

Programming language: JAVA

Constructs:

- Simple Data-type Declaration
- Class Definition
- Simple Object declaration
- Array Declaration
- Looping Constructs

I. COMBINED CODES:

LEXER FILE

```
import ply.lex as lex
```

```
# Token names
```

```
tokens = [
```

```

# Keywords
'CLASS', 'NEW', 'FOR', 'WHILE', 'DO',
'INT', 'FLOAT', 'DOUBLE', 'CHAR_T', 'STRING_T', 'BOOLEAN_T',
'TRUE', 'FALSE',

# Identifiers and literals
'ID', 'INT_LITERAL', 'FLOAT_LITERAL', 'CHAR_LITERAL', 'STRING_LITERAL',

# Operators and punctuation
'EQUAL', 'SEMI', 'COMMA',
'LPAREN', 'RPAREN', 'LBRACE', 'RBRACE',
'LBRACKET', 'RBRACKET',
'PLUSPLUS', 'LT'
]

# Reserved words mapping
reserved = {
    'class': 'CLASS',
    'new': 'NEW',
    'for': 'FOR',
    'while': 'WHILE',
    'do': 'DO',
    'int': 'INT',
    'float': 'FLOAT',
    'double': 'DOUBLE',
    'char': 'CHAR_T',
    'String': 'STRING_T',
    'boolean': 'BOOLEAN_T',
    'true': 'TRUE',
    'false': 'FALSE'
}

# Token regex definitions
t_EQUAL = r'='
t_SEMI = r';'
t_COMMA = r','
t_LPAREN = r'\('

```

```
t_RPAREN = r'\)'
t_LBRACE = r'\{'
t_RBRACE = r'\}'
t_LBRACKET = r'\['
t_RBRACKET = r'\]'
t_PLUSPLUS = r'\+\+'
t_LT = r'<'
```

```
# Literals
```

```
def t_FLOAT_LITERAL(t):
```

```
    r'\d+\.\d+'
```

```
    t.value = float(t.value)
```

```
    return t
```

```
def t_INT_LITERAL(t):
```

```
    r'\d+'
```

```
    t.value = int(t.value)
```

```
    return t
```

```
def t_CHAR_LITERAL(t):
```

```
    r"'([^\n])'"
```

```
    return t
```

```
def t_STRING_LITERAL(t):
```

```
    r'"([^\n"])*'"
```

```
    return t
```

```
# Identifiers & Keywords
```

```
def t_ID(t):
```

```
    r'[A-Za-z_][A-Za-z0-9_]*'
```

```
    t.type = reserved.get(t.value, 'ID')
```

```
    return t
```

```
# Whitespace & line handling
```

```
t_ignore = '\t\r'
```

```
def t_newline(t):
```

```

r'\n+'
t.lexer.lineno += len(t.value)

# Error handling
def t_error(t):
    print(f"❌ Illegal character '{t.value[0]}'")
    t.lexer.skip(1)

# Build the lexer
lexer = lex.lex()

```

PARSER FILE

```

# parser.py
import ply.yacc as yacc
from lexer import tokens

# Global flag for syntax validity
valid = False

# Main rule — allows one statement at a time
def p_statement(p):
    """statement : var_decl
                  | array_decl
                  | object_creation
                  | while_loop
                  | do_while_loop
                  | for_loop
                  | class_def"""
    global valid
    valid = True

# Variable Declarations
def p_var_decl(p):

```

```

"""var_decl : type ID SEMI
    | type ID EQUAL literal SEMI"""
global valid
if len(p) == 6: # with assignment
    declared_type = p[1]
    assigned_value = p[4]
    value_type = p[4][0] if isinstance(p[4], tuple) else None

    if value_type and declared_type != value_type:
        # Handle compatible cases
        if declared_type == 'float' and value_type == 'int':
            valid = True
        elif declared_type == 'double' and value_type in ['float', 'int']:
            valid = True
        else:
            valid = False
            raise SyntaxError(f"Type mismatch in assignment: {declared_type} != {value_type}")
    else:
        valid = True

# Array Declarations (Java-style)
def p_array_decl(p):
    """array_decl : type LBRACKET RBRACKET ID SEMI
        | type LBRACKET RBRACKET ID EQUAL LBRACE literal_list RBRACE SEMI"""
    pass

def p_literal_list(p):
    """literal_list : literal
        | literal_list COMMA literal"""
    pass

# Object Creation
def p_object_creation(p):
    """object_creation : type ID EQUAL NEW ID LPAREN RPAREN SEMI"""
    # Optional: semantic check can be added if types differ
    pass

```

Class Definition

def p_class_def(p):

```
    """class_def : CLASS ID LBRACE class_body RBRACE
                  | CLASS ID LBRACE RBRACE"""
```

pass

def p_class_body(p):

```
    """class_body : class_member
                  | class_body class_member"""
```

pass

def p_class_member(p):

```
    """class_member : var_decl
                  | method_decl"""
```

pass

def p_method_decl(p):

```
    """method_decl : type ID LPAREN param_list_opt RPAREN LBRACE RBRACE"""
```

pass

def p_param_list_opt(p):

```
    """param_list_opt : param_list
                  | empty"""
```

pass

def p_param_list(p):

```
    """param_list : type ID
                  | param_list COMMA type ID"""
```

pass

Loops

def p_while_loop(p):

```
    """while_loop : WHILE LPAREN condition RPAREN LBRACE RBRACE"""
```

pass

def p_do_while_loop(p):

```

    """do_while_loop : DO LBACE RBACE WHILE LPAREN condition RPAREN SEMI"""
    pass

def p_for_loop(p):
    """for_loop : FOR LPAREN var_decl condition SEMI ID PLUSPLUS RPAREN LBACE
    RBACE"""
    pass

# Condition and Literals
def p_condition(p):
    """condition : TRUE
    | FALSE
    | ID
    | literal
    | ID LT literal
    | ID LT ID"""
    pass

def p_literal(p):
    """literal : INT_LITERAL
    | FLOAT_LITERAL
    | CHAR_LITERAL
    | STRING_LITERAL
    | TRUE
    | FALSE"""
    tok = p.slice[1]
    if tok.type == 'INT_LITERAL':
        p[0] = ('int', p[1])
    elif tok.type == 'FLOAT_LITERAL':
        p[0] = ('float', p[1])
    elif tok.type == 'CHAR_LITERAL':
        p[0] = ('char', p[1])
    elif tok.type == 'STRING_LITERAL':
        p[0] = ('string', p[1])
    elif tok.type in ('TRUE', 'FALSE'):
        p[0] = ('boolean', p[1])

```

```

# Type Definitions
def p_type(p):
    """type : INT
        | FLOAT
        | DOUBLE
        | CHAR_T
        | STRING_T
        | BOOLEAN_T"""
    p[0] = p[1].lower()

# Utility rules
def p_empty(p):
    'empty :'
    pass

# Error Handling
def p_error(p):
    global valid
    valid = False
    if p:
        print(f"✗ Syntax error at '{p.value}'")
    else:
        print("✗ Syntax error at EOF")

# Build Parser
parser = yacc.yacc(start='statement')

```

MAIN FILE

```

# main.py
import parser as P
from lexer import lexer

print("✅ Java Syntax Validator — type 'quit' to exit\n")

while True:

```



```

try:
    data = input(">>> ")
except EOFError:
    break
if data.strip().lower() == 'quit':
    break
if not data.strip():
    continue

# reset parse-success flag
P.valid = False

# parse the input (program start allows single or multiple statements)
try:
    P.parser.parse(data, lexer=lexer)
except SyntaxError as e:
    # semantic checks in parser raise SyntaxError
    print(f"❌ {e}")
    P.valid = False
except Exception:
    # p_error prints friendly messages; suppress traceback
    P.valid = False

if P.valid:
    print(f"✅ Accepted — Syntax is correct!\n")
else:
    print(f"❌ Rejected — Invalid syntax or type.\n")

```

II. OUTPUTS:

Simple Data-type Declaration:

```
>>> int a="wrong";  
✗ Syntax error at '"wrong"'  
✗ Rejected – Invalid syntax or type.  
  
>>> float x = 10.5;  
✓ Accepted – Syntax is correct!  
  
>>> char c = 'A';  
✓ Accepted – Syntax is correct!  
  
>>> int;  
✗ Syntax error at ';'   
✗ Rejected – Invalid syntax or type.
```

Class Definition:

```
>>> class Test { int x; float y = 2.5; }  
✓ Accepted – Syntax is correct!  
  
>>> class A {}  
✓ Accepted – Syntax is correct!  
  
>>> class B  
✗ Syntax error at EOF  
✗ Rejected – Invalid syntax or type.  
  
>>> class C {;}  
✗ Syntax error at ';'   
✗ Rejected – Invalid syntax or type.
```

Simple Object declaration:

```

>>> MyClass obj = new MyClass();
✓ Accepted – Syntax is correct!

>>> Car car = new Car();
✓ Accepted – Syntax is correct!

>>> new MyClass();
✗ Syntax error at 'new'
✗ Rejected – Invalid syntax or type.

>>> MyClass obj = MyClass();
✗ Syntax error at 'MyClass'
✗ Rejected – Invalid syntax or type.

>>> MyClass obj = new OtherClass();
✗ Rejected – Invalid syntax or type.

```

Array Declaration:

```

>>> int[] arr = new int[5];
✓ Accepted – Syntax is correct.

>>> int[] arr = {1, 2, 3};
✓ Accepted – Syntax is correct.

>>> int[] arr = {1, 2, 'a'};
✗ Rejected – Invalid syntax or type.

```

Looping Constructs:

```

WARNING: Token 'COMMA' defined, but not used
WARNING: There is 1 unused token
Generating LALR tables
Java Syntax Validator – type 'quit' to exit

>>> for(int i = 0; i < 5; i++) {}
✓ Accepted – Syntax is correct!

>>> while(true){}
✓ Accepted – Syntax is correct!

>>> while(true){
✗ Syntax error at EOF
✗ Rejected – Invalid syntax or type.

>>> do{}while(1)
✗ Syntax error at EOF
✗ Rejected – Invalid syntax or type.

>>> do{}while(false);
✓ Accepted – Syntax is correct!

```

